Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Системы автоматизированного проектирования»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика»
на тему «Бинарное дерево.»

Выполнили:
студенты группы 25BBB4
Волковец А.С
Пантюхин З.С

Приняли:
к.т.н. Юрова О.В.
к.т.н. Деев М.В.

1. # Цель работы

Научиться работать с бинарными деревьями

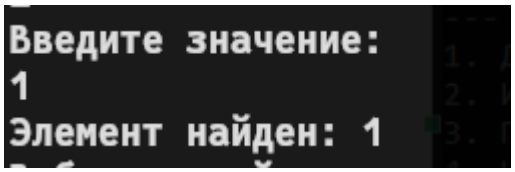
2. # Лабораторное задание

Задание

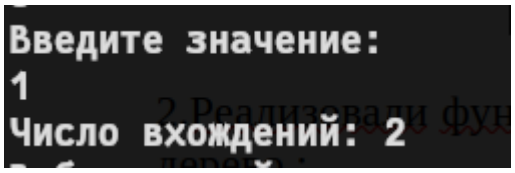
1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. * Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. * Оценить сложность процедуры поиска по значению в бинарном дереве.

3. # Ход работы

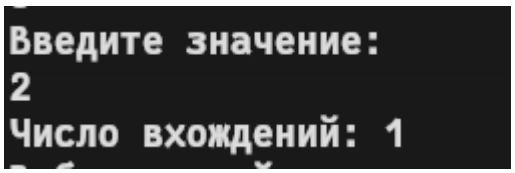
1. Реализовали алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.:



2.Реализовали функцию подсчёта числа вхождений заданного элемента в дерево.:



3.Изменили функцию добавления элементов для исключения добавления одинаковых символов.:



4.Измерили скорость работы алгоритма:
Поиск элемента в Двоичном дереве поиска (BST) основан на свойстве упорядоченности: на каждом шаге мы сравниваем искомое значение (val) с текущим узлом (node->data) и отбрасываем одно из поддеревьев (левое или правое).

В среднем и лучшем случае: $O(\log n)$

Если дерево сбалансировано (или близко к этому), на каждом шаге область поиска уменьшается примерно в два раза (аналогично бинарному поиску в массиве). Если в дереве n элементов, то максимальное количество шагов (глубина дерева) составит $\log_2 n$.

В худшем случае: $O(n)$

Если элементы добавлялись в дерево в отсортированном порядке (например, 1,2,3,4,5), дерево вырождается в линейный список (цепочку). В таком случае алгоритму придется пройти через каждый из n узлов, что дает линейную сложность.

2. Листинг

Файл LAB4.cpp

```
#include <iostream>
#include <limits>
#include <string>
#include <vector>
#include <cmath>

using namespace std;

struct Node {
    int data;
    int count;
    Node* right;
    Node* left;

    Node(int val) : data(val), count(1), left(nullptr), right(nullptr) {}
};

class BinarySearchTree {
private:
    Node* root;

    Node* insert(Node* node, int val) {
        if (node == nullptr) {
            return new Node(val);
        }

        if (val == node->data) {
            node->count++;
        }
        else if (val < node->data) {
            node->left = insert(node->left, val);
        }
        else {
            node->right = insert(node->right, val);
        }
        return node;
    }

    // БЛОК ОТРИСОВКИ
    int getHeight(Node* node) const {
        if (node == nullptr) return 0;
        return 1 + max(getHeight(node->left), getHeight(node->right));
    }
```

```
    void fillMatrix(Node* node, int row, int col, int dist,
vector<vector<string>>& matrix) {
        if (node == nullptr) return;


        string valStr = to_string(node->data);
        matrix[row][col] = valStr;


        if (node->left != nullptr) {
            int nextCol = col - dist;
            int nextRow = row + 2;

            if (row + 1 < matrix.size() && nextCol >= 0) {
                matrix[row + 1][col - dist / 2] = "/";
            }
            fillMatrix(node->left, nextRow, nextCol, dist / 2, matrix);
        }


        if (node->right != nullptr) {
            int nextCol = col + dist;
            int nextRow = row + 2;

            if (row + 1 < matrix.size() && nextCol < matrix[0].size()) {
                matrix[row + 1][col + dist / 2] = "\\";
            }
            fillMatrix(node->right, nextRow, nextCol, dist / 2, matrix);
        }
    }

    void getcount(Node* node, int val) {
        if (node == nullptr) {
            cout << "Элемент не найден" << endl;
            return;
        }
        if (node->data == val) {
            cout << "Число вхождений: " << node->count << endl;
            return;
        }
        if (node->data < val) {
            return getcount(node->right, val);
        }
        else {
            return getcount(node->left, val);
        }
    }

    void find(Node* node, int val) {
        if (node == nullptr) {
            cout << "Элемент не найден" << endl;
            return;
        }
        if (node->data == val) {
            cout << "Элемент найден: " << node->data << endl;
            return;
```

```
        }
        else if (node->data < val) {
            return find(node->right, val);
        }
        else {
            return find(node->left, val);
        }
    }

public:
    BinarySearchTree() : root(nullptr) {}

    void insert_val(int val) {
        root = insert(root, val);
    }


    void print_tree() {
        if (root == nullptr) {
            cout << "Дерево пустое" << endl;
            return;
        }

        int h = getHeight(root);

        int rows = h * 2 - 1;
        int cols = (1 << h) * 4;
        vector<vector<string>> matrix(rows, vector<string>(cols, " "));


        int startCol = cols / 2;
        fillMatrix(root, 0, startCol, cols / 4, matrix);

        cout << "\n--- Структура дерева (классическая) ---\n";
        for (int i = 0; i < rows; i++) {
            bool hasContent = false;

            for (int j = 0; j < cols; j++) {
                if (matrix[i][j] != " ") {
                    hasContent = true;
                    break;
                }
            }
            if (hasContent) {

                for (int j = 0; j < cols; j++) {
                    cout << matrix[i][j];
                }
                cout << endl;
            }
        }
        cout << "----------------------------------------\n" << endl;
    }

    void find(int val) {
        return find(root, val);
    }
```

```
    void getcount(int val) {
        return getcount(root, val);
    }
};

int main() {
    BinarySearchTree bst;
    setlocale(LC_ALL, "rus");
    int choice = 0;
    while (choice != 5) {
        std::cout << "\nВыберете действие:\n";
        std::cout << "1: Добавить элемент\n";
        std::cout << "2: Найти элемент\n";
        std::cout << "3: Найти число вхождений элемента\n";
        std::cout << "4: Отрисовать дерево\n";
        std::cout << "5: Выход\n";
        std::cout << "Ваш выбор: ";

        if (!(std::cin >> choice)) {
            std::cout << "Введено не число\n";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            continue;
        }

        if (choice == 1) {
            int val;
            cout << "Введите значение: ";
            cin >> val;
            bst.insert_val(val);
        }
        else if (choice == 2) {
            int val;
            cout << "Введите значение: ";
            cin >> val;
            bst.find(val);
        }
        else if (choice == 3) {
            int val;
            cout << "Введите значение: ";
            cin >> val;
            bst.getcount(val);
        }
        else if (choice == 4) {
            bst.print_tree();
        }
    }
    return 0;
}
```